Python y bioinformática

Tu guía práctica, paso a paso

Desde crear un archivo **.py** hasta resolver los cinco ejercicios sobre descomposición, dependencias, recursividad y ejecución concurrente.

Esta guía empieza desde cero. Cada sección explica qué hacer, por qué hacerlo y cómo comprobar que funciona. Los ejemplos usan secuencias de ADN como textos formados por A, C, G y T.

Cómo estudiar este PDF

Recorrido	Páginas
Preparar Python y aprender lo básico	2 a 6
Ejercicios 1 y 2: analizar y repartir tareas	7 a 11
Ejercicio 3: medir correctamente	12 y 13
Ejercicio 4: recursividad y dos trabajadores	14 y 15
Ejercicio 5: unir las etapas	16 y 17
Chuleta y revisión final	18 y 19

La idea que debes conservar

Primero consigue un resultado correcto. Después identifica qué tareas son independientes. Finalmente, reparte el trabajo y mide. Dividir un problema o usar recursión no lo ejecuta automáticamente en paralelo.

Qué necesitas

Un editor de texto, una terminal y Python. Los ejemplos solo usan la biblioteca estándar: no necesitas instalar paquetes adicionales. La guía del curso pide Python 3.15 con ejecución free-threaded para estudiar hebras de CPU sin GIL.

Basado en los tres PDF y el ZIP proporcionados. Material didáctico de apoyo a la Guía 2, Unidad II. Incluye supuestos y resultados esperados para practicar.

PASO 01

Crear y ejecutar tu primer archivo

1. Crea una carpeta y un archivo

Crea una carpeta llamada **practica_python**. Abre un editor de texto y guarda un archivo llamado **inicio.py**. Si usas Bloc de notas, selecciona “Todos los archivos” y UTF-8 al guardar. Comprueba que no termine en .py.txt.

2. Escribe una instrucción y guarda

```
print("Hola, estoy aprendiendo Python")
```

print muestra un mensaje. Las comillas indican que el contenido es texto. Los paréntesis contienen lo que le entregas a la instrucción.

3. Abre una terminal en esa carpeta

En Windows, abre la carpeta en el Explorador, escribe **powershell** en su barra de direcciones y pulsa Enter. La terminal sirve para dar órdenes al equipo.

```
python --version
python inicio.py
```

La primera orden muestra la versión. La segunda ejecuta tu archivo. Si tu equipo reconoce **py** en lugar de **python**, puedes usar **py inicio.py**. Si no reconoce ninguno, debes instalar o configurar el intérprete del curso.

4. Separa código de órdenes

Dónde escribo	Qué escribo
Dentro del archivo .py	print("Hola") y las funciones Python
En la terminal	python inicio.py
Si aparece >>>	Estás en el intérprete interactivo. Usa exit() para salir.

Recuerda: Guarda cada modificación antes de volver a ejecutar. A partir de la página 6, reúne las funciones en un archivo llamado **practica.py**. Las funciones van primero; el bloque principal se coloca al final.

PASO 02

Variables, tipos y decisiones

Una variable es un nombre asociado a un valor. Python distingue mayúsculas y minúsculas: **datos** y **Datos** son nombres diferentes.

```
secuencia = "ATGC"   # Texto: str
cantidad = 8         # Entero: int
gc = 50.0            # Decimal: float
cumple = True        # Verdadero o falso: bool
```

El símbolo **#** inicia un comentario. Sirve para explicar el programa y no se ejecuta. Los decimales se escriben con punto.

Símbolo	Significado
= / ==	Asignar un valor / comparar si dos valores son iguales
+ - * /	Sumar, restar, multiplicar y dividir
//	División entera: 7 // 2 da 3
>= <= > <	Mayor o igual, menor o igual, mayor, menor
and / or / not	Y / o / negación
+=	Acumular: total += 1 equivale aquí a total = total + 1

Tomar una decisión con if

```
gc = 50
if gc >= 50:
    print("GC alto")
else:
    print("GC bajo")
```

if significa “si se cumple esta condición”. **else** indica qué hacer en caso contrario. El valor 50 sí cumple, porque usamos mayor **o igual**.

Recuerda: Los dos puntos abren un bloque. Las cuatro posiciones de sangría indican qué líneas pertenecen a él. La sangría es parte de la sintaxis, no una decoración.

PASO 03

Listas, posiciones y repeticiones

Una lista guarda varios elementos en orden. En esta guía, cada elemento suele ser una secuencia completa.

```
datos = ["ATGC", "GGCC", "AT"]
print(len(datos))      # 3 secuencias
print(datos[0])        # ATGC: primera secuencia
print(len(datos[0]))   # 4 bases
print(datos[-1])       # AT: última secuencia
```

Los índices comienzan en cero. **len(datos)** cuenta secuencias; **len(s)** cuenta las letras de una secuencia. No son la misma cantidad.

Recorrer los elementos con for

```
total = 0
for s in datos:
    total += len(s)
print(total)  # 10
```

En cada vuelta, **s** toma un elemento. El acumulador empieza en 0 y pasa por 4, 8 y 10. El último print está fuera del bloque: muestra una sola vez el total final.

Partir una lista en dos

```
medio = len(datos) // 2
izquierda = datos[:medio]
derecha = datos[medio:]
```

En una lista de tres elementos, medio vale 1: queda uno a la izquierda y dos a la derecha. Un corte **datos[inicio:fin]** incluye inicio y excluye fin. Omitir un extremo significa llegar al principio o al final.

Leer las formas cortas del material

```
longitudes = [len(s) for s in datos]
total = sum(len(s) for s in datos)
```

La primera línea construye una lista de longitudes. La segunda las suma. Si estas expresiones te cuestan, usa primero el bucle completo: hace explícito cada paso.

PASO 04

Funciones: entrada, cálculo y salida

Una función agrupa una tarea con nombre. Recibe datos, hace un cálculo y puede devolver una respuesta. Definirla no la ejecuta; tienes que llamarla.

```
def longitud(s):
    return len(s)

resultado = longitud("ATGC")
print(resultado)  # 4
```

Parte	Qué significa
def longitud(s):	Define la función; s es el nombre de la entrada
longitud("ATGC")	Llama a la función con una secuencia concreta
return	Devuelve el resultado y termina esa llamada
print	Muestra información; no reemplaza a return

Diccionarios: guardar resultados con nombre

```
informe = {"gc_alto": 5, "longitud_total": 32}
print(informe["gc_alto"])  # 5
informe["motivo"] = 1
```

Un diccionario relaciona claves con valores. Aquí “gc_alto” es una clave y 5 es su valor. Es útil para entregar varios resultados juntos.

Importar herramientas y comprobar resultados

```
import time
assert longitud("ATGC") == 4
```

import carga una herramienta de la biblioteca. **assert** comprueba una afirmación: si es falsa, detiene la ejecución con AssertionError. Si es verdadera, continúa sin mostrar nada.

Recuerda: En tus comprobaciones, usa ejemplos cuyo resultado puedas calcular a mano. No ejecutes con la opción -O, porque elimina los assert.

PASO 05

Qué calcular: GC, motivos y longitud

Aquí una secuencia válida es un texto no vacío formado solo por **A, C, G y T** en mayúsculas. Por ejemplo, “ATGC”. La validación completa aparecerá en el ejercicio 5.

1. Contenido GC

GC (%) = 100 × (cantidad de G + cantidad de C) / longitud. En ATGC hay 2 bases G/C entre 4 bases: su contenido GC es 50 %.

```
def contenido_gc(s):
    if not s:
        raise ValueError("Secuencia vacía")
    return 100 * (s.count("G") + s.count("C")) / len(s)

def contar_gc_alto(secuencias, umbral=50):
    total = 0
    for s in secuencias:
        if contenido_gc(s) >= umbral:
            total += 1
    return total
```

s.count("G") cuenta letras G. **if not s** detecta un texto vacío; **raise ValueError** comunica un error. “GC alto” cuenta cuántas secuencias cumplen: no suma porcentajes.

2. Apariciones de un motivo y longitud total

```
def contar_motivo(secuencias, motivo="ATG"):
    if not motivo:
        raise ValueError("Motivo vacío")
    return sum(s.count(motivo) for s in secuencias)

def longitud_total(secuencias):
    return sum(len(s) for s in secuencias)
```

Elegimos contar motivos **sin solapamientos**: en AAAA, AA aparece 2 veces; permitiendo solapamientos serían 3. Cuenta dentro de cada secuencia: AT y G en secuencias distintas no forman un ATG.

Recuerda: Una colección vacía produce cero en los tres agregados. Una secuencia vacía no permite calcular GC porque su longitud sería cero.

PASO 06

Ejercicio 1: construir la versión secuencial

Objetivo: obtener los tres resultados en un diccionario. Usa las funciones de la página anterior.

```
def analizar_secuencial(secuencias):
    return {
        "gc_alto": contar_gc_alto(secuencias),
        "motivo": contar_motivo(secuencias),
        "longitud_total": longitud_total(secuencias),
    }
```

Primero, comprueba una entrada pequeña

```
datos = ["ATGC", "GGCC", "ATAT", "CGCG",
         "TTAA", "GCTA", "CCCC", "AATT"]
esperado = {"gc_alto": 5, "motivo": 1,
            "longitud_total": 32}
assert analizar_secuencial(datos) == esperado
```

Cumplen GC alto: ATGC, GGCC, CGCG, GCTA y CCCC. Solo ATGC contiene ATG. Ocho secuencias de cuatro bases suman 32 nucleótidos.

Después, usa el generador de la guía

```
import random

def generar_secuencias(cantidad, longitud, semilla=2026):
    rng = random.Random(semilla)
    bases = "ACGT"
    return [
        "".join(rng.choices(bases, k=longitud))
        for _ in range(cantidad)
    ]
```

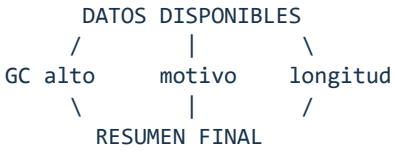
La semilla fija el punto de partida para repetir los datos en el mismo entorno. choices selecciona letras; “”.join las une. El guion bajo indica que no necesitamos el número de la repetición.

Recuerda: La guía usa 20_000 secuencias de longitud 400 como ejemplo. 20_000 y 20000 son el mismo número. Empieza con ocho antes de probar miles.

PASO 07

Ejercicio 1: tareas y dependencias

Una dependencia significa que una tarea necesita un resultado anterior. Una flecha A → B significa que B necesita a A.



Qué responder en los apartados b, c, d y e

b. Grafo: los datos alimentan los tres cálculos. Sus resultados alimentan el resumen final.

c. Independencia: GC, motivo y longitud solo leen la entrada. Ninguno necesita el resultado de otro. El resumen sí necesita los tres.

d. Dos descomposiciones: compara la tabla siguiente.

Estrategia	Ventaja y dificultad
Por funciones: una operación por trabajador	Sencilla y reutiliza las funciones. Las tareas pueden costar distinto: longitud suele terminar antes.
Por datos: un grupo de secuencias por trabajador	Cada grupo ejecuta el mismo análisis. Puede equilibrar mejor la carga, pero hay que repartir y combinar resultados.

e. ¿Independencia garantiza paralelismo? No. La versión secuencial llama una operación después de otra. Necesitas repartir explícitamente el trabajo y disponer de recursos.

Tres palabras que no debes confundir

Concepto	Significado
Concurrencia	Varias tareas progresan en un intervalo, incluso alternándose.
Paralelismo	Varias tareas se ejecutan efectivamente al mismo tiempo.
Recursividad	Una función se llama a sí misma sobre problemas menores.

Recuerda: Compartir datos de solo lectura no obliga a que las tareas se esperen entre sí. Modificar datos compartidos exige coordinación adicional.

PASO 08

Ejercicio 2a: ejecutar con tres hebras

Una **hebra** es un flujo de ejecución dentro de un proceso. Las hebras comparten memoria. Guardaremos cada resultado con su clave en un diccionario compartido.

```
import threading

def guardar_hebra(clave, funcion, datos, salida, candado):
    valor = funcion(datos)
    with candado:
        salida[clave] = valor

def analizar_hebras(secuencias):
    salida = {}
    candado = threading.Lock()
    tareas = [("gc_alto", contar_gc_alto),
              ("motivo", contar_motivo),
              ("longitud_total", longitud_total)]
    hebras = []
    for clave, funcion in tareas:
        h = threading.Thread(
            target=guardar_hebra,
            args=(clave, funcion, secuencias,
                  salida, candado))
        hebras.append(h)
    for h in hebras:
        h.start()
    for h in hebras:
        h.join()
    return salida
```

Lee el código en este orden

1. Se crean tres tareas. **2.** Cada hebra calcula una operación. **3.** Un Lock o candado protege la escritura del diccionario. **4.** Se inician todas. **5.** Se espera a todas antes de devolver el informe.

target recibe el nombre de la función, sin llamarla. **args** contiene sus argumentos. **start()** inicia; **join()** espera, pero no devuelve el resultado de la función.

Recuerda: El cálculo ocurre fuera del candado. Si protegieras todo el cálculo con el mismo candado, las tareas tendrían que ejecutarlo una a una. Este ejemplo supone entradas válidas; join no propaga automáticamente los errores de una hebra.

PASO 09

Ejercicio 2b: ejecutar con tres procesos

Un **proceso** tiene su propio espacio de memoria. Modificar un diccionario ordinario en un hijo no modifica el del padre. Usamos una **Queue** o cola para enviar los resultados.

```
import multiprocessing as mp

def guardar_proceso(clave, funcion, datos, cola):
    cola.put((clave, funcion(datos)))

def analizar_procesos(secuencias):
    cola = mp.Queue()
    tareas = [("gc_alto", contar_gc_alto),
              ("motivo", contar_motivo),
              ("longitud_total", longitud_total)]
    procesos = []
    for clave, funcion in tareas:
        p = mp.Process(target=guardar_proceso,
                       args=(clave, funcion, secuencias, cola))
        procesos.append(p)
        p.start()
    salida = {}
    for _ in procesos:
        clave, valor = cola.get()
        salida[clave] = valor
    for p in procesos:
        p.join()
    cola.close()
    cola.join_thread()
    return salida
```

put envía; **get** recibe y espera si no hay un mensaje disponible. La clave identifica cada resultado: el orden de llegada puede cambiar.

Recuerda: Recibe los mensajes antes de esperar a los procesos. No uses ThreadPoolExecutor ni ProcessPoolExecutor: la guía pide crear y coordinar trabajadores explícitamente.

Supuesto del ejemplo

Las entradas son válidas y los trabajadores terminan correctamente. Si un hijo falla antes de enviar, get puede esperar indefinidamente. Una versión más robusta debe comunicar errores, limitar la espera y cerrar los trabajadores pendientes; no ocultar el fallo.

PASO 10

Ejercicio 2c-f: comprobar y elegir

Protege el inicio del programa

Las funciones anteriores deben estar definidas al nivel del archivo, fuera de main. Coloca este bloque **al final de practica.py**. Al añadir nuevas funciones, ponlas antes del bloque.

```
def main():
    datos = generar_secuencias(2000, 400)
    esperado = analizar_secuencial(datos)
    assert analizar_hebras(datos) == esperado
    assert analizar_procesos(datos) == esperado
    print("Las tres versiones coinciden")

if __name__ == "__main__":
    main()
```

La protección evita que un proceso hijo vuelva a ejecutar el arranque al importar el archivo. Ejecuta desde la terminal: **python practica.py**.

Memoria y sincronización: apartados d y e

```
HEBRAS:   misma lista -> H1, H2, H3 -> diccionario compartido
PROCESOS: datos para P1, P2, P3 -> mensajes -> cola -> padre
```

Si el principal lee antes de terminar, puede encontrar una clave ausente o un resultado incompleto. Espera a las hebras y recibe los resultados de procesos antes de construir el informe.

Elección inicial: apartado f

En Python 3.15 free-threaded **sin GIL activo**, las hebras son una primera opción razonable: comparten datos de solo lectura y evitan transferir la lista a espacios separados. Los procesos ofrecen mayor aislamiento, a cambio de creación y comunicación. La elección final requiere medir.

```
import sys
import sysconfig
print(sys.version)
print(sysconfig.get_config_var("Py_GIL_DISABLED"))
print(sys._is_gil_enabled())
```

Para estudiar hebras sin GIL, verifica soporte de compilación **1** y estado del GIL **False**. La versión por sí sola no basta. Con GIL habilitado no puedes generalizar estos resultados a la condición pedida por la guía.

PASO 11

Ejercicio 3: medir sin engañarte

Objetivo: comparar tiempos completos de las tres versiones. Genera los datos antes de medir; incluye creación, comunicación y espera de trabajadores. Deja impresión y comprobaciones fuera del cronómetro.

```
import time
import statistics

def medir(funcion, datos, esperado, repeticiones=5):
    tiempos = []
    for _ in range(repeticiones):
        inicio = time.perf_counter()
        resultado = funcion(datos)
        duracion = time.perf_counter() - inicio
        assert resultado == esperado
        tiempos.append(duracion)
    return statistics.median(tiempos)

def experimento():
    for cantidad in (2000, 20000, 80000):
        datos = generar_secuencias(cantidad, 400)
        esperado = analizar_secuencial(datos)
        ts = medir(analizar_secuencial, datos, esperado)
        th = medir(analizar_hebras, datos, esperado)
        tp = medir(analizar_procesos, datos, esperado)
        print(cantidad, "secuencial:", ts)
        for nombre, tiempo in [("hebras", th), ("procesos", tp)]:
            speedup = ts / tiempo
            eficiencia = speedup / 3
            overhead = 3 * tiempo - ts
            print(nombre, tiempo, speedup, eficiencia, overhead)
```

Añade estas definiciones antes del bloque principal. Para realizar las mediciones, llama a **experimento()** dentro de main, después de comprobar la equivalencia.

Qué exige el protocolo

Tres cantidades: **2000, 20000 y 80000**; longitud **400**; mínimo **cinco repeticiones** por combinación. La mediana es el valor central al ordenar los cinco tiempos.

Recuerda: Son al menos 45 ejecuciones cronometradas. Usa los mismos datos para todas las versiones y registra versión y GIL. Para reducir sesgos, puedes alternar el orden de las versiones entre repeticiones.

PASO 12

Ejercicio 3: fórmulas y conclusiones

Fórmula	Cómo leerla
S = Ts / Tp	Speedup: cuánto acelera respecto de la versión secuencial.
E = S / p	Eficiencia. Multiplica por 100 para expresarla como porcentaje.
O = p × Tp - Ts	Overhead según la definición de la guía.

Ts es la mediana secuencial; **Tp**, la mediana de la alternativa; **p**, el número de trabajadores. En el ejercicio 3, p = 3. No añadas al principal como un cuarto trabajador.

Ejemplo para entender las fórmulas

Ejemplo hipotético, no una medición: si Ts = 12 s, Tp = 5 s y p = 3: **S = 2,4**; **E = 0,8** (80 %); **O = 3 s**.

Valor de S	Conclusión
Mayor que 1	La alternativa tuvo menor tiempo.
Igual a 1	Los tiempos fueron iguales.
Menor que 1	La alternativa fue más lenta.

Tabla que debes completar para cada tamaño

Versión	Mediana (s)	p	S	E	O (s)
Secuencial	Tu medición	1	1	1	0
Hebras	Tu medición	3	Ts/Th	S/3	3Th-Ts
Procesos	Tu medición	3	Ts/Tp	S/3	3Tp-Ts

Cómo responder los apartados e, f y g

e. Identifica la menor mediana. Si hubo S < 1, explica posibles costos: crear trabajadores, mover datos y esperar tareas desiguales. No atribuyas todo a una causa sin evidencia.

f. Ganar en un tamaño no garantiza escalar: la memoria, la comunicación y las partes secuenciales pueden limitar la mejora.

g. Si el GIL estaba habilitado, escríbelo. p no indica núcleos realmente ocupados y E no mide directamente el uso de CPU. El overhead de la fórmula tampoco aísla el costo de creación.

PASO 13

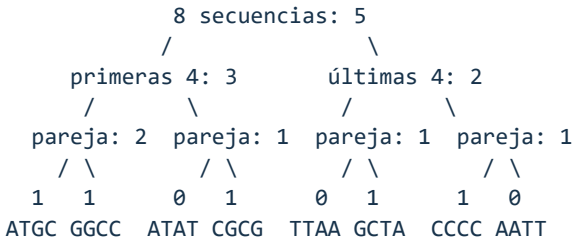
Ejercicio 4a-c: recursión por mitades

Recursividad significa que una función se llama a sí misma. Necesita un caso que termine y llamadas que se acerquen a él.

```
def contar_gc_recursivo(secuencias):
    if len(secuencias) == 0:
        return 0
    if len(secuencias) == 1:
        return int(contenido_gc(secuencias[0]) >= 50)
    medio = len(secuencias) // 2
    izquierda = contar_gc_recursivo(secuencias[:medio])
    derecha = contar_gc_recursivo(secuencias[medio:])
    return izquierda + derecha
```

Lista vacía: devuelve 0. **Una secuencia:** devuelve 1 si cumple y 0 si no; int(True) vale 1. **Más de una:** divide, resuelve ambas mitades y suma.

Árbol de las ocho secuencias del ZIP



Las hojas son los casos de una secuencia. Cada padre suma los valores de sus dos hijos. Las ramas hermanas son independientes; el padre espera ambas. El caso vacío es necesario aunque no aparezca en este árbol.

Por qué todavía no es paralelo

La línea de la izquierda termina antes de ejecutar la de la derecha. Tener ramas independientes permite repartirlas, pero este código aún las ejecuta una tras otra.

Recuerda: En recursión lineal hay una sola llamada hija por paso; se forma una cadena. En la división por mitades aparecen dos ramas que pueden resolverse independientemente.

PASO 14

Ejercicio 4d-g: exactamente dos trabajadores

Divide solo en el nivel superior. Cada trabajador llama a la función recursiva anterior, que continúa de forma secuencial. Reutiliza guardar_hebra y guardar_proceso de las páginas 9 y 10.

```
def contar_gc_paralelo(secuencias):
    medio = len(secuencias) // 2
    grupos = [secuencias[:medio], secuencias[medio:]]
    salida = {}
    candado = threading.Lock()
    hebras = []
    for i in range(2):
        h = threading.Thread(target=guardar_hebra,
            args=(i, contar_gc_recursivo, grupos[i],
                  salida, candado))
        hebras.append(h)
    for h in hebras:
        h.start()
    for h in hebras:
        h.join()
    return salida[0] + salida[1]

def contar_gc_procesos(secuencias):
    medio = len(secuencias) // 2
    grupos = [secuencias[:medio], secuencias[medio:]]
    cola = mp.Queue()
    procesos = []
    for i in range(2):
        p = mp.Process(target=guardar_proceso,
            args=(i, contar_gc_recursivo, grupos[i], cola))
        procesos.append(p)
        p.start()
    salida = dict(cola.get() for _ in range(2))
    for p in procesos:
        p.join()
    cola.close()
    cola.join_thread()
    return salida[0] + salida[1]
```

f. Comprueba que las tres funciones devuelven 5 con la lista del ZIP. La división inicial y la suma final siguen en el principal. En cada trabajador, la recursión es secuencial.

g. Los cortes crean listas nuevas de referencias. Una alternativa es pasar la misma lista e índices inicio/fin. Crear un proceso por llamada multiplicaría costos para tareas muy pequeñas: aquí deben ser exactamente dos.

PASO 15

Ejercicio 5a-b: construir el pipeline

Un **pipeline** organiza el trabajo en etapas. Primero obtienes el lote, luego lo validas, después analizas las secuencias válidas y finalmente construyes el informe.

```
Generar o cargar -> Validar -> GC alto ----\
                            -> Motivo ------> Informe
                            -> Longitud ---/
```

La validación precede a los tres cálculos. Después, esos cálculos son independientes. El informe espera los tres.

```
def validar(secuencias):
    validas = []
    for s in secuencias:
        if s and all(base in "ACGT" for base in s):
            validas.append(s)
    return validas

def pipeline_secuencial(lote):
    validas = validar(lote)
    informe = analizar_secuencial(validas)
    informe["validas"] = len(validas)
    informe["descartadas"] = len(lote) - len(validas)
    return informe

def pipeline_concurrente(lote):
    validas = validar(lote)
    informe = analizar_hebras(validas)
    informe["validas"] = len(validas)
    informe["descartadas"] = len(lote) - len(validas)
    return informe
```

all exige que todas las bases sean A, C, G o T; **s** comprueba que la cadena no esté vacía. En este diseño descartamos también minúsculas y espacios: es una política estricta y explícita.

Recuerda: Puedes elegir normalizar con strip().upper(), pero debes documentarlo y aplicar la misma política a ambas versiones. No cambies los datos de una versión y dejes los de la otra iguales.

PASO 16

Ejercicio 5c-f: justificar y comprobar

c. Justificar la decisión de diseño

Se eligió descomposición funcional con tres hebras porque reutiliza las operaciones y comparte los datos. Para CPU, esta elección se plantea bajo Python 3.15 sin GIL. Ofrece menos aislamiento que procesos y puede no compensar en lotes pequeños. La validación sigue siendo secuencial.

d. Comprobar equivalencia e inválidas

```
lote = ["ATGC", "GGCC", "", "ATXG", "acgt"]
a = pipeline_secuencial(lote)
b = pipeline_concurrente(lote)
assert a == b
assert a["validas"] == 2
assert a["descartadas"] == 3
```

Coloca estas llamadas dentro de main. Prueba además una lista vacía y una lista sin ninguna secuencia válida.

e. Medir al menos dos tamaños

```
for cantidad in (2000, 80000):
    lote = generar_secuencias(cantidad, 400)
    esperado = pipeline_secuencial(lote)
    ts = medir(pipeline_secuencial, lote, esperado)
    tp = medir(pipeline_concurrente, lote, esperado)
    print(cantidad, ts, tp, ts / tp,
          ts / tp / 3, 3 * tp - ts)
```

Usa la función medir de la página 12. Incluye validación, análisis y construcción del informe; deja la generación fuera. Concluye si los datos apoyan la elección inicial.

f. Explicar dos riesgos y sus controles

Riesgo	Cómo controlarlo
Condición de carrera	Cálculos locales; candado solo al escribir; leer después de join.
Bloqueo por trabajador fallido	Enviar errores, limitar la espera y gestionar procesos pendientes.
Desequilibrio de carga	Longitud puede terminar antes; comparar con reparto por grupos.
Demasiados trabajadores	Mantener un número limitado; no crear uno por secuencia.

PASO 17

Chuleta: lo esencial en una página

Necesito…	Escribo o recuerdo…
Ejecutar	python practica.py, desde la terminal
Definir y devolver	def nombre(entrada): y return resultado
Recorrer	for s in secuencias:
Condición de GC alto	contenido_gc(s) >= 50
Porcentaje GC	100 * (s.count("G") + s.count("C")) / len(s)
Dividir en dos	medio = len(datos) // 2; datos[:medio]; datos[medio:]
Recursión correcta	Caso base + problema menor + combinación
Iniciar y esperar	start() inicia; join() espera; inicia todos antes de esperar
Enviar entre procesos	cola.put(...) envía; cola.get() recibe
Comprobar	assert resultado == esperado
Medir	time.perf_counter(); cinco repeticiones y mediana
Fórmulas	S = Ts / Tp; E = S / p; O = p * Tp - Ts

Si algo falla, empieza aquí

Síntoma	Qué revisar
SyntaxError / IndentationError	Dos puntos, paréntesis y sangría.
NameError	Nombre exacto y función definida antes de usarla.
ZeroDivisionError	Hay una secuencia vacía al calcular GC.
KeyError	Clave mal escrita o resultado leído demasiado pronto.
RecursionError	El problema no disminuye o hay demasiada profundidad.
El resultado es None	Falta return; print no devuelve el cálculo.
Se crean procesos sin parar	Falta la protección if __name__ == "__main__".

PASO 18

Revisión antes de entregar la guía

Ejercicio	Debes poder mostrar y explicar
1	Tres operaciones, diccionario secuencial, grafo y comparación por funciones/datos.
2	Tres hebras; tres procesos y cola; assert; memoria compartida/aislada; sincronización y elección.
3	Tres tamaños, longitud 400, cinco repeticiones; medianas, S, E y O; GIL y carga desigual.
4	Casos base y recursivo; árbol de ocho; exactamente dos hebras o dos procesos; cortes e índices.
5	Validación, dos pipelines, equivalencia, dos tamaños, métricas y al menos dos riesgos.

Orden recomendado para tu archivo practica.py

1. Importaciones. **2.** Generador y funciones básicas. **3.** Análisis secuencial. **4.** Trabajadores y versiones concurrentes. **5.** Medición, recursión y pipelines. **6.** Una única función main con las pruebas que quieres ejecutar. **7.** El bloque protegido al final.

Cómo explicar cualquier ejercicio con tus palabras

“La entrada es… La salida debe ser… Primero calculo… Estas tareas son independientes porque… Esta tarea espera porque… Compruebo con… Al medir observé… Mi conclusión se limita a este entorno y estos tamaños.”

Material de origen

Guía 2, Unidad II: **guia-practica(2).pdf**, 5 páginas. Apuntes: **descomposicion (1).pdf**, 30 páginas, y **descomposicion.pdf**, 36 páginas. ZIP: **secuencias.py** y **secuencias-threads.py**. Ambos ejemplos del ZIP deben imprimir 5; sirven como base del ejercicio 4.

Las explicaciones y ejemplos simplificados de este PDF se elaboraron a partir de ese material. Las mediciones de la conversación anterior se hicieron con Python 3.14.7 y GIL habilitado: no se presentan aquí como resultados de Python 3.15 sin GIL.

Documentación oficial de apoyo: free-threading y multiprocessing.

Recuerda: Para aprender, sigue la secuencia: ejemplo pequeño → resultado correcto → dependencias → trabajadores → igualdad → medición. La velocidad se evalúa después de la corrección.